

Machine Learning

Chapter 1: Introduction

Slides based on Tom M. Mitchell, *Machine Learning*
(McGraw-Hill, 1997)

Why Machine Learning?

Well-Posed Learning Problems

Designing a Learning System: Checkers

Perspectives and Issues in Machine Learning

Why Machine Learning?

A field whose time had come

Every day, more of the world's information is generated, sensed, or logged digitally, and much of it is too voluminous, too fast-changing, or too poorly understood for people to hand-code explicit rules to interpret it.

Machine learning studies algorithms that *improve automatically through experience*.

Examples Mitchell opens with:

- Recognizing spoken words
- Predicting recovery rates in patients
- Detecting fraudulent credit-card transactions
- Driving an autonomous vehicle
- Playing world-class backgammon

Why now?

- Growing flood of online, sensor, and scientific data
- Cheap, abundant computation and storage
- Software too complex to hand-craft (face recognition, driving)
- Need for self-customizing software (spam filters, recommenders)
- New scientific insight into human/animal learning

Three vignettes from the book

System	Task	What it learned from
Checkers player (Samuel, 1959)	Choose the best move on a board	Millions of self-play games; reward = win/lose
TD-GAMMON (Tesauro)	Play backgammon at grand-master level	Over 10^6 games of self-play using temporal-difference RL
ALVINN	Steer a van along roads at highway speed	Road images paired with the human driver's steering angle

These examples share a pattern (to be formalized next):

- a **task**,
- a way to **measure performance**, and
- a source of **experience** the program can learn from.

Well-Posed Learning Problems

The defining framework: formalize the pattern

Definition (Mitchell, 1997)

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

To specify a learning problem we must identify three things:

1. T — the **task(s)** the program is meant to perform
2. P — a **performance measure** to evaluate it
3. E — the **training experience** it will learn from

Worked examples of $\langle T, P, E \rangle$

	Task T	Performance P	Experience E
Checkers	Playing checkers	% of games won against opponents	Games played against itself
Handwriting recognition	Recognizing/classifying handwritten words from images	% of words correctly classified	A database of handwritten words with given classifications
Robot driving	Driving on public four-lane highways using vision sensors	Average distance travelled before an error (as judged by a human)	A sequence of images and steering commands recorded while observing a human driver

Designing a Learning System: Checkers

Case study: a checkers-learning program

Mitchell walks through the design of a program that learns to play checkers by playing against itself, in order to make each design choice concrete. Four choices must be made:

1. Choosing the **training experience**
2. Choosing the **target function**
3. Choosing a **representation** for the target function
4. Choosing a **learning algorithm** to approximate the target function

Choice 1: the training experience

Key attributes of training experience:

- **Feedback type** — direct (labeled board $\rightarrow$ best move) vs. **indirect** (only final game outcome is known, so the *credit assignment* problem arises: which of many moves deserves credit for the win?)
- **Degree of control** the learner has over the sequence of training examples (teacher-selected vs. self-selected vs. externally generated)
- How well the **distribution** of training examples represents the distribution over which performance P will ultimately be measured

Choice 1: the training experience (continued)

Design choice made

The program learns from **games played against itself**. This gives an unlimited, cheaply obtained supply of experience, at the cost of an experience distribution that may differ from play against human experts.

Choice 2: the target function

We want a function that lets the program choose its best move from any board state.
Two natural formulations:

$$\text{ChooseMove} : \mathcal{X}_{\text{board}} \rightarrow \mathcal{X}_{\text{move}}$$

$$V : \mathcal{X}_{\text{board}} \rightarrow \mathbb{R}$$

It is far easier to *learn* V , an evaluation function scoring how good a board is, than to learn ChooseMove directly. Ideally:

$$V(b) = 100 \quad \text{if } b \text{ is a final, won board}$$

$$V(b) = -100 \quad \text{if } b \text{ is a final, lost board}$$

$$V(b) = 0 \quad \text{if } b \text{ is a final, drawn board}$$

$$V(b) = V(b') \quad \text{otherwise, where } b' \text{ is the best final board reachable from } b$$

This recursive definition is correct but not efficiently computable — so we must *learn* an approximation $\hat{V}$.

Choice 3: representing $\hat{V}$

We trade expressive power for a representation with few enough parameters to learn from a realistic number of examples. Mitchell chooses a **linear combination of board features**:

$$\hat{V}(b) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

Feature	Meaning
x_1	number of black pieces on the board
x_2	number of red pieces on the board
x_3	number of black kings
x_4	number of red kings
x_5	number of black pieces threatened by red (can be captured next turn)
x_6	number of red pieces threatened by black

Learning $\hat{V}$ now reduces to learning values for the weights $w_0, \dots, w_6$ — a function-approximation problem.

Choice 4: a learning algorithm — estimating training values

We need training examples of the form $\langle b, V_{\text{train}}(b) \rangle$, but we only observe intermediate boards, not their true value. Approximation:

Fact (Training value rule)

$$V_{\text{train}}(b) \leftarrow \hat{V}(\text{Successor}(b))$$

i.e., estimate the value of a board from the learner's *current* estimate of the value of the board that follows it in the game. This works surprisingly well in practice because errors average out over many games, and values propagate backward from the (reliable) end-of-game outcomes.

Choice 4: a learning algorithm — adjusting the weights (LMS)

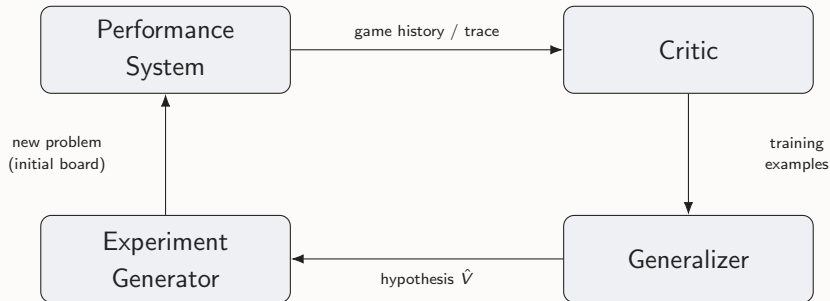
Fit the weights so that $\hat{V}$ best matches the training values, minimizing squared error $E \equiv \sum_{\langle b, V_{\text{train}}(b) \rangle} (V_{\text{train}}(b) - \hat{V}(b))^2$. Mitchell uses the **Least Mean Squares (LMS)** rule, applied incrementally to each training example:

Algorithm 1: LMS weight-update procedure (η : small learning rate, $x_0 \equiv 1$)

```
1 foreach training example  $\langle b, V_{\text{train}}(b) \rangle$  do
2   use current weights to compute  $\hat{V}(b)$ ;
3   for  $i \leftarrow 0$  to 6 do
4      $w_i \leftarrow w_i + \eta (V_{\text{train}}(b) - \hat{V}(b)) x_i$ 
```

Intuitively: when $\hat{V}(b)$ under-estimates $V_{\text{train}}(b)$, every weight is nudged up in proportion to its feature's contribution, and vice versa.

The final design: four modules



- **Performance System:** solves the task (plays a game) using the current $\hat{V}$
- **Critic:** analyzes the trace and produces labeled training examples
- **Generalizer:** runs the learning algorithm (LMS) to update $\hat{V}$
- **Experiment Generator:** proposes the next problem (opening board) to explore, e.g., to improve coverage

Perspectives and Issues in Machine Learning

Learning = searching a hypothesis space

A useful perspective: learning is a search through a large space of candidate hypotheses $\mathcal{H}$ (here, all possible weight vectors) for the one that best fits the training examples and any prior constraints. Different learning algorithms differ chiefly in:

- the **hypothesis space** they search (e.g., linear functions, decision trees, neural nets)
- the **search strategy** used to move through it (e.g., gradient descent, greedy specialization)

This view unifies the very different algorithms studied in later chapters.

Key open questions

- What algorithms exist for learning general target functions from specific training examples? When do they converge? Which perform best for which problem types?
- How much training data is sufficient? What general bounds relate confidence in learned hypotheses to training-set size?
- When and how can *prior knowledge* held by the learner guide the process of generalizing from examples?
- What is the best strategy for choosing a useful next training experience, and how does that choice alter the complexity of the learning problem?
- What is the best way to reduce the learning task to one or more function-approximation problems?
- How can the learner automatically alter its representation to improve its ability to represent and learn the target function?

Chapter summary

- Machine learning develops algorithms that improve with experience, formalized by the $\langle T, P, E \rangle$ framework.
- Designing a learning system requires choosing the training experience, the target function, a representation for it, and a learning algorithm.
- The checkers example shows how an intractable exact target function (V) can be replaced by a learnable approximation ($\hat{V}$), fit with an incremental gradient-style rule (LMS).
- Learning can be cast generally as a search through a hypothesis space — the theme that organizes the rest of the book: concept learning, decision trees, neural networks, Bayesian methods, computational learning theory, instance-based methods, genetic algorithms, rule learning, analytical learning, and reinforcement learning each explore this theme with a different $\mathcal{H}$ and search strategy.